# Bit Manipulations : 3

**Agenda:**
1. Count set bits
2. Single number
3. Single element 2 (every element 3 times & 1 unique element)
4. Single number 3

## ① Count set bits → Goal to count how many bit s = 1

**M-1**

```
ans = 0
for i → 0 to 31 {        ↗ long = 63
    if (N & (1 << i) > 0) {
        ans ++
    }
}
return ans
```

→ TC = O(32) = O(1)

↳ SC = O(1)

**M-2**

```
ans = 0
while (N > 0) {
    ans += (N & 1)        // check if last bit is 1.
    N = N >> 1            // N/2 → rightshift to check next bit
}
return ans
```

## ② Single number 1

Problem: every number in an array occurs twice except for one unique number.

↳ By XORing all the elements in the array, the pairs cancel each other out (0), leaving only the uniquenum.

**Code** →

```
public int SN (int[] nums) {
    int ans = 0
    for (int num : nums) {
        ans ^= num;
    }
    return ans;
}
```

## ③ Single number II

↳ every number occurs 3 times except for 1 unique number.

- **Approach :- Bit Counting**

Code →

```
public int sing (int[] nums){
    int ans =0
    for (int i=0; i<32 ; i++){
        int cnt =0;
        for (int num : nums){
            if ((num >> i) & 1 == 1){
                cnt ++;
            }
        }
        if (cnt % 3 != 0){
            ans |= (1<<i);
        }
    }
    return ans;
}
```

Check every bit position

or (num & (1<<i)) != 0

↳ Logic:

| Count of 1's at bit i | | Meaning |
|---|---|---|
| $3x$ | → | only triplets contributed |
| $3x+1$ | → | unique number has 1 |
| $3x+2$ | → | impossible (Problem guarantees only 1 unique) |

| Bit Position | Count of 1s | Count % 3 | Result Bit |
|---|---|---|---|
| 0 (LSB) | 1 | 1 | 1 |
| 1 | 4 | 1 | 1 |
| 2 | 0 | 0 | 0 |

011 = 3

## (4) Single Number III

↳ every number occurs twice except for 2 unique numbers (x and y)

Step: 1 : XOR of entire array

Step: 2 : find a bit where x and y differ

Step: 3 : Divide numbers int 2 groups based on diffBit
Grp 1 → bit is 1.
Grp 0 → bit is 0

~~Step: 4~~

---

Code:

```
public class Sol. {
    public int[] solve (int[] A) {
        int xorAll = 0;
        for (int num : A) {
            xorAll ^= num;
        }

        int diffBit = 0;
        for (int i = 0 ; i < 32 ; i++ ) {
            if ((xorAll & (1<<i)) > 0) {
                diffBit = i;
                break;
            }
        }

        int num1 = 0;
        int num0 = 0;
        for (int num : A) {
            if ((num & (1<<diffBit)) > 0) {
                num1 ^= num; }
            else {
                num0 ^= num ; }
            }

        return new int [] {
            Math.min (num1, num0),
            Math.max (num1, num0)
            };
    }
}
```